Lockbox Parquet Refresh

What this folder is, why it exists, and how to change it
Victor, Transfers/Operations

What this does

The Lockbox Power BI reports used to read the Manager Report Detail CSVs directly. They now read Parquet files instead. After new CSVs are added, run refresh.bat before refreshing either report in Power BI Desktop. It converts both source folders and usually finishes in well under a minute.

Source CSVs → Parquet folders used by the reports

```
Lockbox 12month raw data → Lockbox 12month raw data (parquet)
Current Raw Data → Current Raw Data (parquet)
```

Why the old version was so slow

There were two separate problems. The smaller one was that the reports parsed about 235 MB of CSV data on every refresh, including around 30 columns that were loaded, assigned types, and then immediately discarded.

The bigger problem was five calculated columns whose processing time grew with the square of the row count. Each used logic like this:

```
CALCULATE(MAX([EarliestBatch]),
FILTER(Table, [CREATEDBYUSER] = EARLIER([CREATEDBYUSER])
&& [EarliestBatch] < EARLIER([EarliestBatch])))
```

The goal is simple: find the previous batch for each user. But DAX calculated columns do not have a built-in concept of "the row before this one," so this formula scans the table again for every row. With roughly 3 million rows, that works out to about 9 trillion row comparisons per column, repeated across five columns. A refresh took around ten minutes, used all available memory, and failed completely on a 16 GB machine.

The roughly 390 measures in the model were not causing the problem. Measures only run when a visual needs them, so they do not add much to refresh time. Calculated columns and calculated tables are different because they are calculated and stored during refresh. Adding more RAM helped the refresh finish, but it did not solve the underlying problem. The calculation itself was simply too expensive.

How the fix works

lockbox_to_parquet.py now does in Python what those calculated columns used to do in DAX. Instead of repeatedly scanning the table, it sorts the data and uses grouped shifts to find previous rows. This takes seconds instead of minutes.

All 28 calculated columns across the two fact tables were moved into the Python script and removed from the Power BI model. They now come into Power BI as normal loaded columns with the same names, so the existing measures still work. Loaded columns also compress better than calculated columns, which makes the model smaller.

The script also removes the roughly 30 unused columns before writing the Parquet files. It handles the employer-name and module spelling cleanup that used to happen in Power Query, and it reads holidays.csv for the Hold Duration calculation.

Two things that may not be obvious

1. The whole table is processed at once.

"Previous batch for this user" can refer to a batch from the previous month's file. Because of that, the script reads all months together, calculates the derived columns, and then splits the data back into separate monthly files.

This means adding one new month rebuilds all of the monthly Parquet files. That is intentional. Processing each file separately would give the first batch of every month the wrong previous-batch value.

2. Two bugs from the original version were intentionally fixed.

A blank previous timestamp used to turn into about 1,104,586 hours instead of staying blank because DAX treats BLANK() as 1899-12-30. This threw off averages.

A Hold row with no problem date could also end up with a duration of about 45,969 days. That triggered the "Hold Duration >= 4" condition and incorrectly labeled those rows as Stop Pay Pending.

If you need to reproduce the old numbers exactly for comparison, set BUG_COMPAT_MODE = True near the top of the script.

Making changes later

If you need to add or remove a source column:

Edit DROP_COMMON in the script, then add or remove the same column name from the Power Query SelectColumns list for that table. The two need to match.

If you need to change a spelling cleanup:

Edit EMPLOYERNAME_FIXES_MONTHLY, MODULE_FIXES, or TRANTYPE_FIXES_WEEKLY. The two reports currently clean some data differently. That behavior came from the original reports and was kept as-is.

If you need to change a derived column:

Find the matching function in the script. The functions are named after the original DAX columns. Make the change there rather than recreating the column as a calculated column in Power BI.

If you need to add a holiday:

Add a row to holidays.csv. The Power BI Holiday Table reads the same file, so the Python calculations and Power BI stay in sync.

To rebuild everything:

```
refresh.bat --force
```

To check that the CSV schemas match:

```
refresh.bat --audit
```

The main rule is simple: if a column name changes in the script, it also needs to change in Power BI, and vice versa.

Power BI connects measures to columns by name. If a column is renamed upstream, Power BI sees the old column as deleted and the renamed column as something new. Any measures using the old name will break on the next refresh. That type of mismatch is what caused the original outage that led to this work.

Files in this folder:

```
refresh.bat: run this
lockbox_to_parquet.py: does the conversion and calculations
holidays.csv: NYSE closures from 2020 through 2040
```

Python, pandas, and pyarrow are required. refresh.bat installs pandas and pyarrow the first time it runs and prints instructions if Python is not installed.